

Operating Systems Laboratory

Process Management — C Programs

3rd Semester CSE (AI & ML) • Simple programs with correct sample outputs

Learning objective: Understand PID, PPID, process creation, parent-child relationships, program replacement using `exec()`, and parent-child synchronization using `wait()`.

Basic terms

Program: instructions stored in a file. Process: a program in execution. PID: process ID. PPID: parent process ID. Parent: process that creates a child. Child: newly created process.

These programs require Linux/POSIX. Run them in Ubuntu, WSL 2, or an Ubuntu virtual machine.

1. Process ID — getpid() and getppid()

C Program

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    // Get the PID of the current process
    printf("Current Process PID = %d\n", getpid());

    // Get the PID of the parent process
    printf("Parent Process PPID = %d\n", getppid());

    return 0;
}
```

Compile and execute

```
gcc pid.c -o pid
./pid
```

Sample/Correct output:

Sample output (PID values vary on every run):
Current Process PID = 2451
Parent Process PPID = 2300

How to understand: The numbers are examples. Your actual PID/PPID will be different.

2. Create a Child Process — fork()

C Program

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int pid;

    // Create a new child process
    pid = fork();

    if (pid < 0)
    {
        printf("Process creation failed\n");
    }
    else if (pid == 0)
    {
        // This block runs in the child
        printf("Child: PID = %d\n", getpid());
    }
    else
    {
        // This block runs in the parent
        printf("Parent: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

Compile and execute

```
gcc fork_demo.c -o fork_demo
./fork_demo
```

Sample/Correct output:

Possible output (PID values vary):
Parent: PID = 2451
Child PID = 2452
Child: PID = 2452

How to understand: Output order can vary because the Linux scheduler decides which runnable process runs first.

3. Parent Waits for Child — fork() + wait()

C Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid;

    // Create a child process
    pid = fork();

    if (pid < 0)
    {
        printf("Process creation failed\n");
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child: PID = %d\n", getpid());
        printf("Child is working...\n");
    }
    else
    {
        // Parent waits for child to finish
        printf("Parent: waiting for child...\n");
        wait(NULL);
        printf("Parent: child has completed.\n");
    }

    return 0;
}
```

Compile and execute

```
gcc wait_demo.c -o wait_demo
./wait_demo
```

Sample/Correct output:

```
Correct sequence (PID values vary):
Parent: waiting for child...
Child: PID = 2452
Child is working...
Parent: child has completed.
```

How to understand: wait(NULL) makes the parent wait until the child finishes.

4. Replace the Child Program — fork() + execlp()

C Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid;

    // Create a child process
    pid = fork();

    if (pid == 0)
    {
        // Child replaces its program with the 'ls' command
        execlp("ls", "ls", "-l", NULL);

        // Runs only if execlp() fails
        printf("execlp failed\n");
    }
    else if (pid > 0)
    {
        // Parent waits for the child
        wait(NULL);
        printf("Parent: child completed.\n");
    }
    else
    {
        printf("fork failed\n");
    }

    return 0;
}
```

Compile and execute

```
gcc child_exec.c -o child_exec
./child_exec
```

Sample/Correct output:

```
Possible output:
total 8
-rwxr-xr-x 1 student student ... child_exec
-rw-r--r-- 1 student student ... child_exec.c
Parent: child completed.
```

How to understand: The exact file sizes, permissions and filenames in the ls output vary. The important point is that the child executes ls instead of continuing the original child code.

5. Two Child Processes — Multiple fork()

C Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int p1, p2;

    // Create first child
    p1 = fork();

    if (p1 == 0)
    {
        printf("Child 1: PID = %d\n", getpid());
        return 0;
    }

    // Parent creates second child
    p2 = fork();

    if (p2 == 0)
    {
        printf("Child 2: PID = %d\n", getpid());
        return 0;
    }

    // Parent waits for both children
    wait(NULL);
    wait(NULL);

    printf("Parent: both children completed.\n");

    return 0;
}
```

Compile and execute

```
gcc two_children.c -o two_children
./two_children
```

Sample/Correct output:

Possible output (PID values and child order vary):
Child 1: PID = 2452
Child 2: PID = 2453
Parent: both children completed.

How to understand: The parent creates two children. wait(NULL) is called twice so the parent waits for both child processes.

Important Notes for Students

- PID and PPID values are dynamic; they will not match the sample numbers exactly.
- After fork(), parent and child are separate processes.
- The output order of parent/child programs may vary because of CPU scheduling.
- exec() replaces the current process image with another program; it does not create a second process by itself.
- wait() allows a parent to synchronize with a child.
- Compile with gcc and execute with ./program.

Useful Linux commands

```
ps
ps -f
ps -ef
echo $$
pstree
```

Student practice

- Run the PID program three times and compare the PIDs.
- Run fork() and identify parent and child.
- Run fork()+wait() and explain the execution order.
- Run exec() and identify which program the child executes.
- Run the two-child program and draw the process tree.

These are introductory programs for 3rd-semester CSE (AI & ML) students. More advanced topics can be introduced after these programs are understood.